

MOBILE WEB SERVICE MODULE

CONTENTS

This module has the following files:

1. MOBILE WEB SERVICE MODULE.docx
2. MOBILE WEB SERVICE MODULE.pdf
3. MOBILE WEB SERVICE MODULE MEDIA.zip

INTRODUCTION

The client has requested the development of a fast, mobile-first web app called **ApaKabar**, an **Online Media Platform**. The primary goal is to deliver a smooth and accessible experience, especially for users on mobile devices. Performance and ease of access are top priorities, as the main target audience consists of mobile users.

PROJECT OVERVIEW

Your task is to build a **Mobile-first Web Application** using a JavaScript framework of your choice either **React or Vue**. Alternatively, you may build it using **Vanilla JavaScript (Native)** without any framework. A **pre-built API is provided** to support your development. These are the following core features:

- Fully optimized for mobile screens only
- Home page with personalized news based on selected categories
- Discover page with search, pagination and category filters
- The article detail page displays all attributes of the selected article.
- Bookmark feature for saving articles
- Light, dark, and system appearance modes

MOBILE WEB LAYOUT

Screen Size: Max width of 480px (mobile screen only).

The layout consist of three main sections:

- Header Bar (Top)
- Main Content (Scrollable)
- Navigation Bar (Bottom)

The header bar is fixed at the top of the screen which displays the current page title and an optional back button for navigation.

The main content is positioned between the header and navigation bars. It must be scrollable when content exceeds the viewport height.

The navigation bar is fixed at the bottom of the screen.

NAVIGATION BAR

There are 4 buttons in the navigation bar:

1. Home
2. Discover
3. Bookmark
4. Settings

THE DATA API

API Endpoint provided on <http://api-mobile-service.lksn.id>

The API includes the following main endpoints:

- Get all posts
 - GET /api/v1/posts
- Get a post by slug
 - GET /api/v1/posts/:slug
- Get all categories
 - GET /api/v1/categories

QUERYING POSTS

The `/api/v1/posts` endpoint supports several optional query parameters to filter and paginate the results:

- `page` — number, min: 1, default: 1 (e.g. 5 for page 5)
- `per_page` — number, min: 1, default: 20 (e.g. 50 to show 50 posts)
- `order_by` — latest or popular, default: latest
- `search` — string, default: empty ("") to reset search
- `category` — comma-separated category slugs (e.g. `slug_1,slug_2`)

HOME PAGE

The first button in the navigation bar opens the Home page.

BREAKING NEWS

This section highlights the urgent news articles.

Breaking news items are shown in a horizontally scrollable list with scroll snapping, allowing users to swipe through stories smoothly one by one.

RECOMMENDATION LIST

These recommendations are articles based on the user's selected category preferences and aim to show content that matches their interests.

DISCOVER PAGE

The second button in the navigation bar opens the Discover page.

This page allows users to explore news articles.

SEARCH

Users can search for articles using keywords. The search input is designed to improve performance and reduce unnecessary API calls by using a debounce mechanism.

- The app waits until the user has stopped typing for 0.5–1 second before sending a request to the API.
- It means that search does **not** trigger an API request on every keystroke or on clicking any button.

FILTER BY CATEGORY

The category filter allows users to filter news articles based on specific categories. The list of categories is fetched dynamically from the API.

INFINITE SCROLL

When the user scrolls to the bottom, the web page should load further article data and display further articles to the users. This is also known as infinite scrolling.

Please fine-tune the timing and smoothness of infinite scrolling. The loading of further content should be not too late, and not loading too early. In other words, the infinite scroll should not feel laggy, and should not eagerly load too much data. Please ensure the data loading is correct. That means there should not have multiple instances of the same record displayed, and not missing any records to display.

ARTICLE DETAIL PAGE

The Article Detail Page displays the full content of a selected news article.

It shows all relevant information, including:

- Category name
- Title
- Publish date
- Author name
- Visited count
- Thumbnail or cover image
- Full content or body text
- Tags

At the bottom of the page, there should be a section that displays 3 related news articles that from the same category as the current article and exclude the article currently being viewed

BOOKMARK

The third button in the navigation bar opens the Bookmark view.

This page displays a list of articles that the user has saved for later reading.

Users can add or remove bookmarks directly from the article detail page or from the article list view.

SETTINGS

The fourth button in the navigation bar opens the Settings view.

THEME MODE

Users can choose between three theme options: light, dark and system (follow the device's light/dark setting)

CATEGORY PREFERENCE

Users can select or unselect their preferred news categories. These preferences will be used to personalize the Recommendation section on the Home page.

OFFLINE MODE

When users lose internet connection while using the app, a dedicated offline page (game.html) is displayed to maintain a user experience. The html file is provided in media files.

MEDIA FILES

- Wireframes – visual guides for layout and UI reference
- JavaScript Frameworks – React/Vue (with Vite setup)
- HTML file for offline page

INSTRUCTIONS FOR COMPETITORS

- You must configure the API base URL in a single place, such as:
 - Using an environment variable:
 - `VITE_API_URL=http://api-mobile-service.lksn.id` (in a .env file)
 - Or defining a constant in your code:
 - `const apiUrl = 'http://api-mobile-service.lksn.id'`
- Please configure the **manifest.json** and essential meta tags for both Android and iOS mobile web systems.
- The viewport should also be configured for mobile web friendly viewing.
- Please ensure having good accessibility. We will assess the Chrome Lighthouse accessibility score.
- Please also ensure creating easy-to-maintain JavaScript code.

PROJECT STRUCTURE & SUBMISSION

1. Rename project folder to: **XX_MOBILE_WEB_SERVICE_MODULE** where **XX** is your computer number.
2. If you are using NodeJs Framework, please place the **build version** of your project to **XX_MOBILE_WEB_SERVICE_MODULE**, so your project can be accessible from **index.html**
3. Zip the root folder/your project (not your build version) into a single file **XX_MOBILE_WEB_SERVICE_MODULE.zip**.

Exclude node_modules when zipping the project (if exists).

4. Upload your **XX_MOBILE_WEB_SERVICE_MODULE** and **XX_MOBILE_WEB_SERVICE_MODULE.zip** to the FTP server inside "submission" folder.